

Week 2: Agent with Tools and Memory (smolagents)

Objective: Build an agent that can search the web and remember conversation context, using a framework instead of hand-rolling the tool-calling loop.

Steps:

1. Install [smolagents](#), get an HF token (free tier)

```
PS D:\OTHER\IntelligentSystemsLab> pip install "smolagents[toolkit]" "smolagents[transformers]"
Downloading fsspec-2026.6.0-py3-none-any.whl (203 kB)
Downloading lxml-6.1.1-cp313-cp313-win_amd64.whl (4.0 MB)
 4.0/4.0 MB 10.0 MB/s 0:00:00
Downloading markdownify-1.2.3-py3-none-any.whl (15 kB)
Downloading primp-1.3.1-cp310-abi3-win_amd64.whl (4.7 MB)
 4.7/4.7 MB 11.3 MB/s 0:00:00
Downloading transformers-5.13.0-py3-none-any.whl (11.5 MB)
 11.5/11.5 MB 12.9 MB/s 0:00:00
Downloading tokenizers-0.22.2-cp39-abi3-win_amd64.whl (2.7 MB)
 2.7/2.7 MB 11.7 MB/s 0:00:00
Downloading regex-2026.6.28-cp313-cp313-win_amd64.whl (277 kB)
Downloading safetensors-0.8.0-cp310-abi3-win_amd64.whl (355 kB)
Downloading accelerate-1.14.0-py3-none-any.whl (389 kB)
Downloading torch-2.13.0-cp313-cp313-win_amd64.whl (122.1 MB)
 122.1/122.1 MB 15.1 MB/s 0:00:08
Downloading networkx-3.6.1-py3-none-any.whl (2.1 MB)
 2.1/2.1 MB 19.1 MB/s 0:00:00
Downloading sympy-1.14.0-py3-none-any.whl (6.3 MB)
 6.3/6.3 MB 15.6 MB/s 0:00:00
Downloading mpmath-1.3.0-py3-none-any.whl (536 kB)
 536.2/536.2 kB 15.4 MB/s 0:00:00
Downloading brotli-1.2.0-cp313-cp313-win_amd64.whl (369 kB)
Downloading torchvision-0.28.0-cp313-cp313-win_amd64.whl (4.2 MB)
 4.2/4.2 MB 14.7 MB/s 0:00:00
Downloading types-0.26.8-py3-none-any.whl (122 kB)
```

Access Tokens

User Access Tokens

[+ Create new token](#)

Access tokens authenticate your identity to the Hugging Face Hub and allow applications to perform actions based on token permissions. **Do not share your Access Tokens with anyone;** we regularly check for leaked Access Tokens and remove them immediately.

You have no Access Token

2. Build a basic cloud agent with no tools, run one task, observe output

Use Cloud Model

```

from smolagents import CodeAgent, InferenceClientModel

model = InferenceClientModel(
    model_id="meta-llama/Llama-3.3-70B-Instruct",
    token="YOUR_HF_TOKEN"
)

agent = CodeAgent(tools=[], model=model)

result = agent.run("Calculate the sum of numbers from 1 to 50")
print(result)

```

✓ 21.0s Python

tion.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages\tqdm\auto.py:21: TqdmWarning: IProgress not found. Please use `from tqdm.auto import tqdm` in Python scripts.

New run

Calculate the sum of numbers from 1 to 50

InferenceClientModel - meta-llama/Llama-3.3-70B-Instruct

Step 1

— Executing parsed code: —

```

sum_of_numbers = sum(range(1, 51))
print(sum_of_numbers)

```

Execution logs:

1275

Out: None

3. Add `DuckDuckGoSearchTool()`, run a task that needs current info and requires two or more steps (e.g. "search for X, then calculate Y from it"), so the loop visibly repeats. Watch the **Thought** → **Code** → **Observation** cycle live

[**Note:** Some of you may encounter rate-limit by `DuckDuckGoSearchTool()` when many students from the same lab IP access this tool. In that case, please feel free to explore other web search APIs such as Brave Search API. The following web resource provides a useful summary of available web search tools: <https://www.kdnuggets.com/7-free-web-search-apis-for-ai-agents>]

```

from smolagents import CodeAgent, InferenceClientModel, DuckDuckGoSearchTool

agent = CodeAgent(
    tools=[DuckDuckGoSearchTool()],
    model=model
)

result = agent.run("Search for the current population of Melbourne, then calculate 15% of that number.")
print(result)

```

✓ 11.9s Python

New run

Search for the current population of Melbourne, then calculate 15% of that number.

InferenceClientModel - meta-llama/Llama-3.3-70B-Instruct

Step 1

— Executing parsed code: —

```

population_melbourne = web_search(query="current population of Melbourne")
print("Population of Melbourne:", population_melbourne)

```

Execution logs:

Population of Melbourne: ## Search Results

[Melbourne - Wikipedia](https://en.wikipedia.org/wiki/Melbourne)
 4 days ago - The name is also used specifically ... on the Melbourne central business district and some im
 surrounds. The city occupies much of the northern and eastern coastlines of Port Phillip Bay. As of June 2
 population of the city was 5.435 million, or 19% of the population of Australia. It is currently the south
 ...

[Demographics of Melbourne - Wikipedia](https://en.wikipedia.org/wiki/Demographics_of_Melbourne)
 1 week ago - The cities of Whittlesea, Wyndham, Hume, Brimbank and Dandenong on Melbourne's fringe are par
 current migrant hotspots. As a result of large migrant populations, Melbourne has a proliferation of areas
 restaurants, cafes and services of similar international demographic establish, ...

[Melbourne Population 2026](https://worldpopulationreview.com/cities/australia/melbourne)
 1 week ago - Melbourne's 2026 population is now estimated at 4,928,443. In 1996, the population of Melbourn
 2,875,627. Melbourne has grown by 68,581 in the last year, which represents a 1.41% annual change. These pop
 values and projections come ...

```

Step 2

- Executing parsed code: -----
population_melbourne = 5465000
fifteen_percent = population_melbourne * 0.15
print("15% of the population of Melbourne:", fifteen_percent)
final_answer(fifteen_percent)

Execution logs:
15% of the population of Melbourne: 819750.0

Final answer: 819750.0

[Step 2: Duration 1.68 seconds | Input tokens: 5,118 | Output tokens: 174]

819750.0

```

4. Inspect `agent.memory.steps` to see what the framework tracked automatically

```

for step in agent.memory.steps:
    print(step)
    print("---")

TaskStep(task='Search for the current population of Melbourne, then calculate 15% of that number.', task_images=None)
---
ActionStep(step_number=1, timing=Timing(start_time=1784162373.6376202, end_time=1784162376.2017214, duration=2.564101219177246), model_id='Qwen/Qwen2.5-1.5B-Instruct')
---
ActionStep(step_number=2, timing=Timing(start_time=1784162376.201855, end_time=1784162377.2256553, duration=1.0238003730773926), model_id='Qwen/Qwen2.5-1.5B-Instruct')
---

```

5. Swap `InferenceClientModel` (cloud) for `TransformersModel` (**local**), same agent code, run the same task

Use Local Model

```

# Use Local Model agent

from smolagents import CodeAgent, TransformersModel, DuckDuckGoSearchTool

model = TransformersModel(model_id="Qwen/Qwen2.5-1.5B-Instruct")

agent = CodeAgent(
    tools=[DuckDuckGoSearchTool()],
    model=model
)

result = agent.run("Search for the current population of Melbourne, then calculate 15% of that number.")
print(result)

```

* The local model may take a long time to download and run, and it may fail as well.

6. Compare: same task, cloud vs local backend, no code change beyond the model line

```
# Setup cloud vs local agents
# Use verbosity_level=LogLevel.ERROR to suppress log

cloud_model = InferenceClientModel(
    model_id="meta-llama/Llama-3.3-70B-Instruct",
    token="YOUR_HF_TOKEN"
)
cloud_agent = CodeAgent(
    tools=[DuckDuckGoSearchTool()],
    model=cloud_model,
    verbosity_level=LogLevel.ERROR
)

local_model = TransformersModel(model_id="Qwen/Qwen2.5-1.5B-Instruct")
local_agent = CodeAgent(
    tools=[DuckDuckGoSearchTool()],
    model=local_model,
    verbosity_level=LogLevel.ERROR
)

import time

def run_and_time(agent, task):
    start = time.time()
    result = agent.run(task)
    elapsed = time.time() - start
    return result, elapsed

task = "What is the current weather in Melbourne?"

cloud_result, cloud_time = run_and_time(cloud_agent, task)
local_result, local_time = run_and_time(local_agent, task)

print(f"Cloud: {cloud_time:.2f}s\n{cloud_result}\n")
print(f"Local: {local_time:.2f}s\n{local_result}")
```

7. Wrap the agent in [GradioUI](#) for an interactive chat demo

```

PS D:\OTHER\IntelligentSystemsLab> pip install 'smolagents[gradio]' 'gradio==5.9.1'
Downloading pillow-11.1.0-cp313-cp313-win_amd64.whl.metadata (9.3 kB)
Collecting smolagents[gradio]
  Downloading smolagents-1.12.0-py3-none-any.whl.metadata (14 kB)
  Downloading smolagents-1.11.0-py3-none-any.whl.metadata (14 kB)
  Downloading smolagents-1.10.0-py3-none-any.whl.metadata (14 kB)
  Downloading smolagents-1.9.2-py3-none-any.whl.metadata (14 kB)
  Downloading smolagents-1.9.1-py3-none-any.whl.metadata (14 kB)
  Downloading smolagents-1.9.0-py3-none-any.whl.metadata (14 kB)
  Downloading smolagents-1.8.1-py3-none-any.whl.metadata (14 kB)
  Downloading smolagents-1.8.0-py3-none-any.whl.metadata (14 kB)
  Downloading smolagents-1.7.0-py3-none-any.whl.metadata (10.0 kB)
  Downloading smolagents-1.6.0-py3-none-any.whl.metadata (9.7 kB)
  Downloading smolagents-1.5.1-py3-none-any.whl.metadata (9.7 kB)

```

```

from smolagents import CodeAgent, InferenceClientModel, DuckDuckGoSearchTool, GradioUI

model = InferenceClientModel(
    model_id="meta-llama/Llama-3.3-70B-Instruct",
    token="hf_dSsmvSIyLGdIlHmLYEzTVxCBVAshfFKmi"
)

agent = CodeAgent(tools=[DuckDuckGoSearchTool()], model=model)
GradioUI(agent).launch()

```

⊗ 0.0s

Checkpoint questions:

- What did the framework do for you that you would have written manually in Week 1?
- How many Thought → Code → Observation cycles ran before the agent gave a final answer, and what triggered it to stop?
- What changed when you swapped from cloud to local, and what stayed the same?
- Did your local model complete the task successfully? If not, what did it get wrong?

Deliverable: Notebook with a working agent (web search tool + memory), run against both a cloud and local model, plus the same agent running in a GradioUI chat interface.